NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur

Affiliated to Pokhara University

LAB REPORT FOR EMBEDDED SYSTEMS

notes.arpanadhikari7.com.np

LAB REPORT 7: VHDL CODE FOR DE-MULTIPLEXER

Submitted by:

Name: Arpan Adhikari

Roll No.: 231309

BE-CE (5th SEM)

Submitted to:

Er. Simanta Kasaju

Available on notes.arpanadhikari7.com.np

S. N	LAB TOPIC	DATE	REMARKS
1.	VHDL Code (OR & AND Gate)	20th Nov, 2025	
2.	VHDL Code (OR & NOT Gate) using testbench	23rd Nov	
3.	Universal Gates and Special Gates	30th Nov	
4.	VHDL Code for Adder	7th Dec	
5.	VHDL Code for Subtractor	14th Dec	
6.	VHDL Code for Multiplexer	21st Dec	
7.	VHDL Code for De-Multiplexer	4th Jan, 2026	

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

LAB 7: VHDL CODE FOR DE-MULTIPLEXER

OBJECTIVE:

The objective of this lab is to understand the working principle of Demultiplexers by designing and implementing 1:4 and 1:8 Demultiplexer circuits using VHDL. This experiment aims to familiarize students with combinational logic design, select line operation, and VHDL coding techniques for data distribution circuits. It also helps in simulating and verifying the correct operation of demultiplexers for different input and select line combinations.

THEORY:

Introduction to Demultiplexer

A Demultiplexer (DEMUX) is a combinational digital circuit that takes a single input signal and routes it to one of several output lines based on the values of select lines. It performs the reverse operation of a multiplexer and is widely used in digital communication systems, data distribution, and control applications.

1:4 DEMULTIPLEXER

A **1:4 Demultiplexer** distributes one input data line to one of four output lines using two select lines.

Inputs

- Data input: D
- Select lines: S1, S0

Outputs

- Y0, Y1, Y2, Y3

Working Principle

notes.arpanadhikari7.com.np

Based on the binary value of the select lines, the input data is transferred to the corresponding output line, while the remaining outputs stay at logic zero.

Logic Equations

- $Y0 = D \cdot \overline{S}1 \cdot \overline{S}0$
- $Y1 = D \cdot \overline{S}1 \cdot S0$
- $Y2 = D \cdot S1 \cdot \overline{S}0$
- $Y3 = D \cdot S1 \cdot S0$

Truth Table

S1	S0	Active Output
0	0	Y0
0	1	Y1
1	0	Y2
1	1	Y3

Available on notes.arpanadhikari7.com.np

1:8 DEMULTIPLEXER

A **1:8 Demultiplexer** routes a single input data line to one of eight output lines using three select lines.

Inputs

- Data input: D
- Select lines: S2, S1, S0

Outputs

- Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7

Working Principle

Each unique combination of select lines activates only one output line, allowing the input data to appear at that output.

Logic Equations

$$Y0 = D \cdot \overline{S2} \cdot \overline{S1} \cdot \overline{S0}$$
$$Y1 = D \cdot \overline{S2} \cdot \overline{S1} \cdot S0$$
$$Y2 = D \cdot \overline{S2} \cdot S1 \cdot \overline{S0}$$
$$Y3 = D \cdot \overline{S2} \cdot S1 \cdot S0$$
$$Y4 = D \cdot S2 \cdot \overline{S1} \cdot \overline{S0}$$
$$Y5 = D \cdot S2 \cdot \overline{S1} \cdot S0$$
$$Y6 = D \cdot S2 \cdot S1 \cdot \overline{S0}$$
$$Y7 = D \cdot S2 \cdot S1 \cdot S0$$

Truth Table

S2	S1	S0	Active Output
0	0	0	Y0
0	0	1	Y1
0	1	0	Y2
0	1	1	Y3
1	0	0	Y4
1	0	1	Y5
1	1	0	Y6
1	1	1	Y7

notes.arpanadhikari7.com.np

Demultiplexer Implementation Using VHDL

VHDL (VHSIC Hardware Description Language) is used to model demultiplexer circuits using concurrent signal assignments or case statements. The select lines determine which output receives the input data. Simulation of the VHDL code helps verify the correct functionality of the demultiplexer for all possible input and select combinations.

Available on notes.arpanadhikari7.com.np

1:4 DEMUX

DATAFLOW FOR 1:4 DEMUX:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arpdeMux is
port(
  I : in std_logic;
  S : in std_logic_vector(1 downto 0);
  Y : out std_logic_vector(3 downto 0)
);
end arpdeMux;

architecture arch_arpdeMux of arpdeMux is
begin
  Y(0) <= I and not S(0) and not S(1);
  Y(1) <= I and     S(0) and not S(1);
  Y(2) <= I and not S(0) and     S(1);
  Y(3) <= I and     S(0) and     S(1);
end arch_arpdeMux;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

BEHAVIORAL FOR 1:4 DE-MUX:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arpdeMux is
port(
  I : in std_logic;
  S : in std_logic_vector(1 downto 0);
  Y : out std_logic_vector(3 downto 0)
);
end arpdeMux;

architecture arch_arpdeMux of arpdeMux is
begin
  process (I,S)
    begin
     Y <= "0000";

    case S is
    when "00" => Y(0) <= I;
    when "01" => Y(1) <= I;
    when "10" => Y(2) <= I;
    when "11" => Y(3) <= I;
    when others => Y <= "0000";
    end case;
  end process;
end arch_arpdeMux;
```


```
library IEEE;
use IEEE.std_logic_1164.all;

entity arpdeMux is
port(
  I : in std_logic;
  S : in std_logic_vector(1 downto 0);
  Y : out std_logic_vector(3 downto 0)
);
end arpdeMux;

architecture arch_arpdeMux of arpdeMux is
begin
  process (I,S)
    begin
     Y <= "0000";

     case S is
     when "00" => Y(0) <= I;
     when "01" => Y(1) <= I;
     when "10" => Y(2) <= I;
     when "11" => Y(3) <= I;
     when others => Y <= "0000";
     end case;
   end process;
end arch_arpdeMux;
```


notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

TESTBENCH FOR 1:4 DE-MUX:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_arpdemux is
end entity;

architecture arch_demuxarp of test_arpdemux is

component arpdeMux
port(
  I : in std_logic;
  S : in std_logic_vector(1 downto 0);
  Y : out std_logic_vector(3 downto 0)
);
end component;

signal I : std_logic;
signal S : std_logic_vector(1 downto 0);
signal Y : std_logic_vector(3 downto 0);

begin
UUT: arpdeMux port map
(
I => I,
S => S,
Y => Y
);

stim_proc: process
begin

   I <= '0';
  S <= "00"; wait for 100 ps;
  S <= "01"; wait for 100 ps;
  S <= "10"; wait for 100 ps;
  S <= "11"; wait for 100 ps;
  I <= '1';
  S <= "00"; wait for 100 ps;
  S <= "01"; wait for 100 ps;
  S <= "10"; wait for 100 ps;
  S <= "11"; wait for 100 ps;
end process;
end arch_demuxarp;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_arpdemux is
end entity;

architecture arch_demuxarp of test_arpdemux is

component arpdeMux
port(
  I : in std_logic;
  S : in std_logic_vector(1 downto 0);
  Y : out std_logic_vector(3 downto 0)
);
end component;

signal I : std_logic;
signal S : std_logic_vector(1 downto 0);
signal Y : std_logic_vector(3 downto 0);

begin
UUT: arpdeMux port map
(
I => I,
S => S,
Y => Y
);

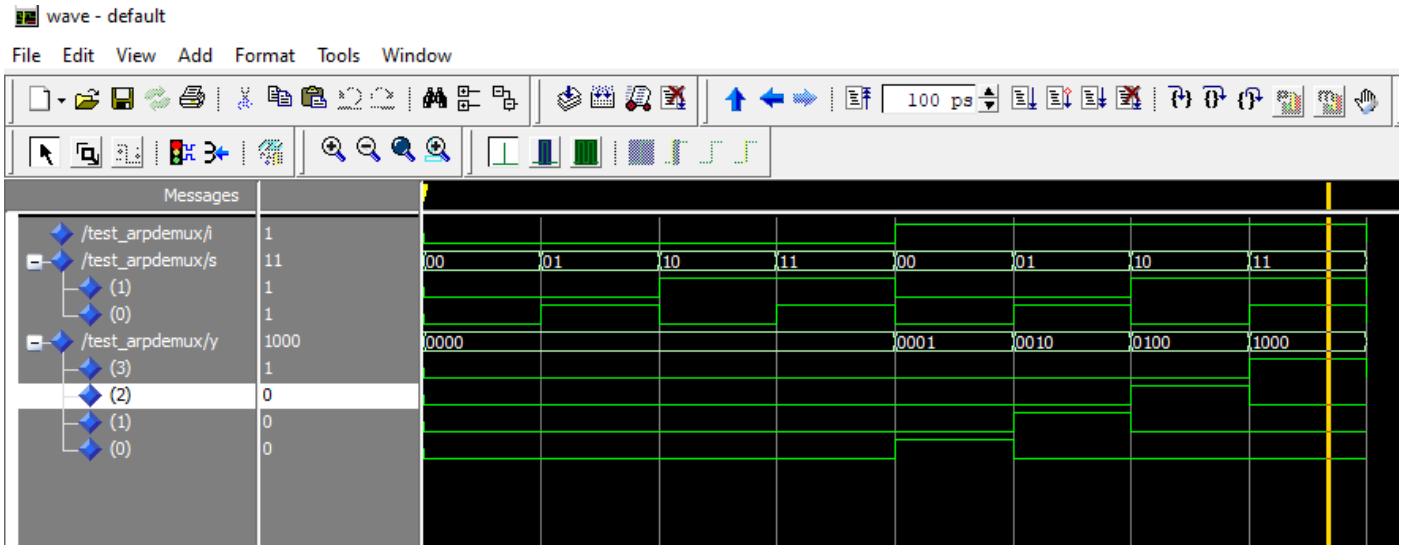
stim_proc: process
begin

   I <= '0';
  S <= "00";
  wait for 100 ps;
  S <= "01";
  wait for 100 ps;
  S <= "10";
  wait for 100 ps;
  S <= "11";
  wait for 100 ps;
  I <= '1';
  S <= "00";
  wait for 100 ps;
  S <= "01";
  wait for 100 ps;
  S <= "10";
  wait for 100 ps;
  S <= "11";
  wait for 100 ps;
end process;
end arch_demuxarp;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

WAVEFORM FOR 1:4 DEMUX:



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1:8 DEMUX

DATAFLOW FOR 1:8 DEMUX:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arpdeMux is
port(
  I : in std_logic;
  S : in std_logic_vector(2 downto 0);
  Y : out std_logic_vector(7 downto 0)
);
end arpdeMux;

architecture arch_arpdeMux of arpdeMux is
begin
  Y(0) <= I and (not S(0)) and (not S(1)) and (not S(2));
  Y(1) <= I and (not S(0)) and (not S(1)) and (    S(2));
  Y(2) <= I and (not S(0)) and (    S(1)) and (not S(2));
  Y(3) <= I and (not S(0)) and (    S(1)) and (    S(2));
  Y(4) <= I and (    S(0)) and (not S(1)) and (not S(2));
  Y(5) <= I and (    S(0)) and (not S(1)) and (    S(2));
  Y(6) <= I and (    S(0)) and (    S(1)) and (not S(2));
  Y(7) <= I and (    S(0)) and (    S(1)) and (    S(2));
end arch_arpdeMux;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

BEHAVIORAL FOR 1:8 MUX:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arpdeMux is
port(
  I : in std_logic;
  S : in std_logic_vector(2 downto 0);
  Y : out std_logic_vector(7 downto 0)
);
end arpdeMux;

architecture arch_arpdeMux of arpdeMux is
begin
  process (I,S)
    begin
     Y <= "00000000";

     case S is
      when "000" => Y(0) <= I;
     when "001" => Y(1) <= I;
     when "010" => Y(2) <= I;
     when "011" => Y(3) <= I;
     when "100" => Y(4) <= I;
     when "101" => Y(5) <= I;
     when "110" => Y(6) <= I;
     when "111" => Y(7) <= I;
     when others => Y <= "00000000";
     end case;
   end process;
end arch_arpdeMux;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;

entity arpdeMux is
port(
  I : in std_logic;
  S : in std_logic_vector(2 downto 0);
  Y : out std_logic_vector(7 downto 0)
);
end arpdeMux;

architecture arch_arpdeMux of arpdeMux is
begin
  process (I,S)
    begin
     Y <= "00000000";

     case S is
      when "000" => Y(0) <= I;
     when "001" => Y(1) <= I;
     when "010" => Y(2) <= I;
     when "011" => Y(3) <= I;
     when "100" => Y(4) <= I;
     when "101" => Y(5) <= I;
     when "110" => Y(6) <= I;
     when "111" => Y(7) <= I;
     when others => Y <= "00000000";
     end case;
   end process;
end arch_arpdeMux;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

TESTBENCH FOR 1:8 DEMUX:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_arpdemux is
end entity;

architecture arch_demuxarp of test_arpdemux is

component arpdeMux
port(
  I : in std_logic;
  S : in std_logic_vector(2 downto 0);
  Y : out std_logic_vector(7 downto 0)
);
end component;

signal I : std_logic;
signal S : std_logic_vector(2 downto 0);
signal Y : std_logic_vector(7 downto 0);

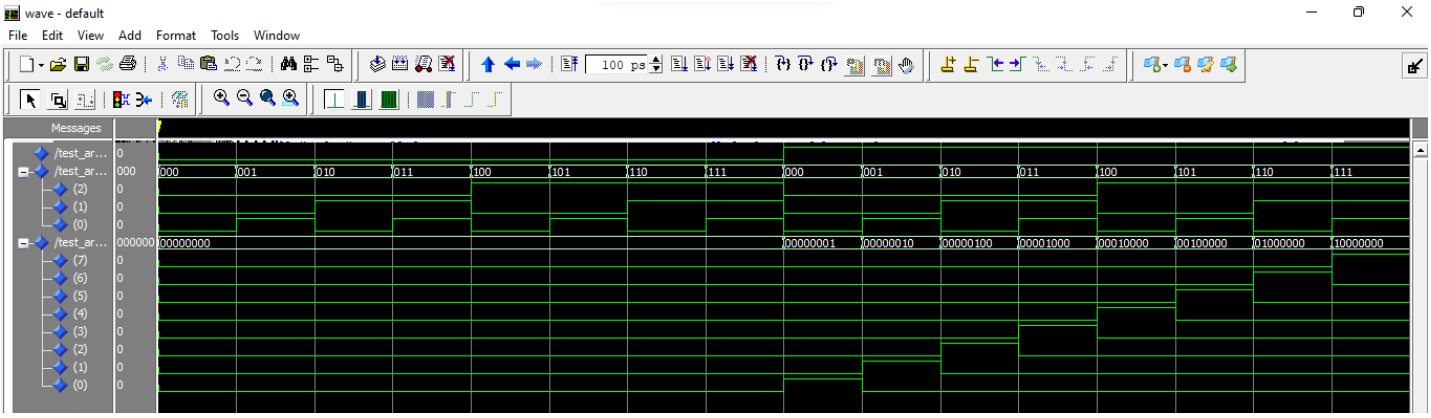
begin
UUT: arpdeMux port map
(
I => I,
S => S,
Y => Y
);

stim_proc:process
    begin
       I<='0';
      S<="000"; wait for 100 ps;
      S<="001"; wait for 100 ps;
      S<="010"; wait for 100 ps;
      S<="011"; wait for 100 ps;
      S<="100"; wait for 100 ps;
      S<="101"; wait for 100 ps;
      S<="110"; wait for 100 ps;
      S<="111"; wait for 100 ps;
      I<='1';
      S<="000"; wait for 100 ps;
      S<="001"; wait for 100 ps;
      S<="010"; wait for 100 ps;
      S<="011"; wait for 100 ps;
      S<="100"; wait for 100 ps;
      S<="101"; wait for 100 ps;
      S<="110"; wait for 100 ps;
      S<="111"; wait for 100 ps;
    end process;
end arch_demuxarp;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

WAVEFORM FOR 1:8 DEMUX:



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

RESULT:

The 1:4 and 1:8 Demultiplexer circuits were successfully designed and implemented using VHDL. Simulation was performed for all possible combinations of select lines and input data. The results showed that the input data was correctly directed to the selected output line, while all other output lines remained at logic zero. The observed simulation waveforms matched the theoretical truth tables, confirming the correct operation of both demultiplexer circuits.

CONCLUSION:

In this experiment, the functionality of 1:4 and 1:8 Demultiplexers was successfully studied and verified using VHDL. The experiment improved understanding of data distribution controlled by select lines and demonstrated the importance of demultiplexers in digital systems. The simulation results were in complete agreement with theoretical expectations, proving the correctness of the VHDL implementation. This experiment provides a strong foundation for implementing data routing and control logic in complex digital designs.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np